teuscher-lab.com
Christof Teuscher
ECE 361
Fall 2026
Portland State UNIVERSITY

Homework 5
Computer System Organization
ECE 361
Fall 2026

Overview

Homework 5 is a finite state machine written as a table: a combination lock. You turn a description into states, events, and a transition table, then into C where one table lookup replaces a chain of `if`s. It is due on **Sun, Nov 1, 11:59pm**, pushed to `hw05/`. AI tools are allowed. Your Shine about your code is due by **Wed, Nov 4, 11:59pm**, without any supporting materials. Satisfactory / Not Yet, as before.

The lock

- The code is **3-1-4**, typed on a keypad. Each right digit moves on; the third one opens the door (UNLOCK).
- A wrong digit beeps (BEEP) and starts over from the beginning, even in the middle of the code.
- While a code is being typed, a 5-second timer runs; if it runs out, the lock starts over, silently.
- While open, digits do nothing. Closing the door, or the timer running out, locks it again (LOCK).
- Closing the door while it is already locked, or while a code is being typed, changes nothing.

Part 1: The table

In `hw05/lock.h`, use exactly these types; add a comment per function:

```
typedef enum { LOCKED, GOT1, GOT2, OPEN, N_STATES } state_t;
typedef enum { E_RIGHT, E_WRONG, E_CLOSE, E_TIMEOUT, N_EVENTS } event_t;
typedef enum { A_UNDEFINED, A_NONE, A_UNLOCK, A_LOCK, A_BEEP, N_ACTIONS } action_t;
typedef struct { state_t next; action_t action; } transition_t;

extern const transition_t lock_table[N_STATES][N_EVENTS];
state_t     lock_step(state_t s, event_t e, action_t *action);
int         lock_event(state_t s, char c);
const char *state_name(state_t s);     /* also event_name, action_name */
```

- **lock_table in lock.c:** all 16 entries, written with designated initializers (`[GOT1] = { ... }`). Draw the state diagram and the table on paper first.
- **lock_step:** one table lookup, no `if` or `switch` on the state. **lock_event:** turns a character into an event for state `s`: a digit is `E_RIGHT` if it is the next digit of the code, otherwise `E_WRONG`; `c` is the door closing, `t` the timer running out; anything else −1.
- **The names** come from `const char *` arrays indexed by the enum; `"?"` if out of range.

Part 2: A program, tests, and a Makefile

- `hw05/main.c`: starts in `LOCKED`, reads standard input character by character, and prints one line per event: the state, the character, the event, the next state, and the action, e.g. `GOT2 4 RIGHT -> OPEN UNLOCK`; at the end, `final: STATE`. Try it on a `data/codes.txt` of your own.
- `hw05/tests/test_lock.c` tests, at least: every one of the 16 table entries; `lock_event` for a right and a wrong digit in each state, while open, and for `c`, `t`, and other characters; and the sequences 3 1 4 (opens), 3 3 1 4 (does not), 3 1 t 4 (does not), and 3 1 4 c (locks again). PASS or FAIL per test, a summary, a nonzero exit status on failure.

- `hw05/Makefile`: `make` builds `lock` with no warnings, `make test` builds and runs the tests, `make clean`. Each `.o` lists the headers it includes.
- `hw05/README.md`: how to build and test, and your state diagram and transition table (a table in Markdown; the diagram in ASCII or a photo of your drawing).

Optional bonus: a table checker (one revision token)

In `hw05/check.c` and `check.h`, write `int fsm_check(const transition_t table[N_STATES][N_EVENTS], state_t start, FILE *out)`. It prints one line per problem and returns how many it found: an entry left out (`A_UNDEFINED`, which is why it is 0), a next state out of range, a state that cannot be reached from `start` (search breadth first over the table), and a state with no way out. Add `tests/test_check.c`, a `make bonus` target that runs it, and `BONUS.md`: your checker's output on three broken copies of your table, and why each is a real bug for a lock. A good answer to one extra Shine question about it earns the token.

Your project: a next step (not graded)

Scaffolding, not graded and not on the Shine: after Thursday's studio, turn your device's transition table into `project/fsm.h` and `fsm.c` the same way: an `enum` for your states and your events, and a table indexed by both.

Deliverable, Shine, and requirements

- **Push by Sun, Nov 1, 11:59pm, in hw05/:** `lock.h`, `lock.c`, `main.c`, `data/`, `tests/test_lock.c`, `Makefile`, `README.md`, `AI_USAGE.md`; for the bonus, also `check.h`, `check.c`, `tests/test_check.c`, `BONUS.md`.
- **Shine:** opens Mon, Nov 2, in the morning; due Wed, Nov 4, 11:59pm; 10 to 15 minutes about your own code, with questions like "Why does 3 3 1 4 not open, and which table entry would you change so that it does?" No notes, no second screen, no AI, no other person; webcam on.
- **For Satisfactory:** every deliverable at the deadline; `make` with no warnings, `make test` passes, `make clean` works; `lock_step` is one table lookup; your Shine completed and meeting the bar.